ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN

BÀI TẬP LỚN

Môn: CS112 – Phân tích và thiết kế thuật toán

ĐỀ TÀI

DEMO ĐIỀU PHỐI CẢNG CHO HÃNG TÀU

Thuê bến (tô màu đồ thị) · Phân công đội · Quỹ giờ công (cái túi) · Xếp container (đóng thùng)

Giảng viên: **Huỳnh Thị Thanh Thương**

Sinh viên thực hiện: **Võ Thị Thanh Tuyền**

MSSV: **[điền MSSV]**

Lớp: **[điền lớp]**

Mã nguồn: **github.com/SerenaVo-code/berth-contract-coloring (thư mục replication/app)**

Tháng 10 năm 2026

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU BÀI TOÁN....4
1.1 Bối cảnh....4
1.2 Lý do chọn bài toán....4
1.3 Ứng dụng thực tế....5
CHƯƠNG 2. MÔ TẢ BÀI TOÁN....6
2.1 Bài toán 1: Thuê bến (tô màu đồ thị khoảng)....6
2.2 Bài toán 2: Phân công đội....6
2.3 Bài toán 3: Quỹ giờ công (cái túi 0/1)....6
2.4 Bài toán 4: Xếp container (đóng thùng)....6
2.5 Liên hệ giữa các module....6
CHƯƠNG 3. MÔ HÌNH BÀI TOÁN....7
3.1 Module 1: Thuê bến....7
3.2 Module 2: Phân công đội....7
3.3 Module 3: Quỹ giờ công (cái túi 0/1)....8
3.4 Module 4: Xếp container (đóng thùng)....8
CHƯƠNG 4. THIẾT KẾ CẤU TRÚC DỮ LIỆU VÀ THUẬT TOÁN....9
4.1 Cấu trúc dữ liệu chung và cách sinh bước....9
4.2 Module 1: Thuê bến....9
4.3 Module 2: Phân công đội....11
4.4 Module 3: Quỹ giờ công (cái túi 0/1)....12
4.5 Module 4: Xếp container (đóng thùng)....13
4.6 Tính đúng đắn và cách đánh giá chất lượng....14
CHƯƠNG 5. VÍ DỤ MINH HOẠ CÁC BƯỚC THỰC HIỆN THUẬT TOÁN....15
5.1 Dữ liệu và đồ thị xung đột....15
5.2 Tham lam theo giờ đến....15
5.3 Tham lam chọn hợp đồng rẻ nhất....15
5.4 DSATUR....16
5.5 Quay lui có cắt nhánh....17
5.6 Kết quả tổng hợp....17
CHƯƠNG 6. TƯỜNG THUẬT QUÁ TRÌNH THIẾT KẾ VÀ XÂY DỰNG....19
6.1 Công cụ và phạm vi sử dụng AI....19
6.2 Dòng thời gian....19
6.3 Các prompt đã sử dụng....20
6.4 Vấn đề gặp phải và cách giải quyết....20
6.5 Kiểm chứng, chỉnh sửa và phát triển tiếp kết quả....21
6.6 Tài liệu prompt dựng lại theo framework (bổ trợ)....21
6.7 Bản dựng lại bằng AI (bản nộp)....21
CHƯƠNG 7. THIẾT KẾ THỰC NGHIỆM....23
7.1 Mục tiêu....23
7.2 Cách tạo testcase....23
7.3 Phân nhóm và kích thước....23
7.4 Cách đảm bảo bao phủ các trường hợp....24

7.5 Cách chạy 24
CHƯƠNG 8. ĐÁNH GIÁ 25
8.1 Tính đúng đắn: kết quả chạy testcase 25
8.2 Chất lượng của các thuật toán 25
8.3 Thời gian thực hiện 26
8.4 Giao diện và khả năng minh hoạ từng bước 26
8.5 Hạn chế 26
8.6 Hướng phát triển 27
8.7 Tổng hợp mối liên hệ và kết luận 27
TÀI LIỆU THAM KHẢO 28
PHỤ LỤC A. HƯỚNG DẪN CHẠY VÀ CẤU TRÚC MÃ NGUỒN 29
A.1 Chạy trên Codespaces hoặc trên máy 29
A.2 Cấu trúc thư mục 29
PHỤ LỤC B. NHẬT KÝ PROMPT (PHIÊN 12–13/10/2026) 30
PHỤ LỤC C. TÀI LIỆU PROMPT DỰNG LẠI THEO FRAMEWORK 32

CHƯƠNG 1. GIỚI THIỆU BÀI TOÁN

1.1 Bối cảnh

Một hãng tàu container khai thác nhiều chuyến tàu mỗi ngày. Mỗi chuyến cập cảng đều kéo theo một chuỗi quyết định rời rạc: thuê bến nào, giờ nào; đội bốc xếp nào phục vụ; dùng quỹ giờ công tăng ca cho những yêu cầu dịch vụ nào; và xếp các lô container lên tàu ra sao. Các quyết định này đều là bài toán tối ưu tổ hợp, và có thể mô hình hoá bằng những bài toán kinh điển trong môn Phân tích và thiết kế thuật toán.

Đồ án xây dựng **một demo có giao diện gồm bốn bài toán trong cùng một khung giao diện**: chọn thuật toán, nhập dữ liệu, xem thuật toán chạy từng bước và so sánh kết quả. Mỗi bài toán đều có nhiều thuật toán thuộc các chiến lược thiết kế khác nhau (tham lam, quy hoạch động, quay lui có cắt nhánh), để so sánh được chất lượng lời giải và thời gian.

Module	Bài toán thực tế	Mô hình kinh điển	Thuật toán trong demo
1. Thuê bến	Thuê ít tiền nhất cho n tàu, mỗi tàu cần một bến trong khoảng [đến, rời]	Tô màu đồ thị khoảng	Tham lam theo giờ đến, tham lam chọn hợp đồng rẻ nhất, DSATUR, quay lui
2. Phân công đội	Gán đội đủ kỹ năng cho từng lượt tàu, không trùng giờ	Gán việc có ràng buộc (tô màu có danh sách cho phép)	Tham lam theo giờ đến, tham lam tàu khó trước, quay lui + cận
3. Quỹ giờ công	Chọn các yêu cầu tăng ca trong quỹ giờ có hạn	Cái túi 0/1	Tham lam theo giá trị/giờ, quy hoạch động, quay lui + cận
4. Xếp container	Xếp các lô lên ít tàu nhất	Đóng thùng (bin packing)	First-Fit, First-Fit giảm dần, Best-Fit giảm dần, quay lui + cận

Bảng 1.1. Bốn module của demo

1.2 Lý do chọn bài toán

Em chọn bài toán từ ví dụ **bài toán Sở thú (tô màu đồ thị)** mà cô đã trình bày trong các buổi học. Đây là dạng cô nhấn mạnh trong lưu ý của đề: bài toán tô màu đồ thị cần được mô hình hoá và thiết kế thuật toán cho đúng. Em muốn đưa bài toán này ra khỏi dạng bản đồ quen thuộc để gắn với một bối cảnh có chi phí cụ thể, nên em chuyển nó sang tình huống **thuê bến tàu** nhìn từ phía hãng tàu: n tàu cần bến, mỗi hợp đồng thuê bến (một màu) phục vụ được nhiều tàu không trùng giờ, và mục tiêu là tiền thuê nhỏ nhất.

Để bài toán không dừng ở bản cơ bản, em mô hình chi phí hợp đồng gồm **phí cố định F cho mỗi hợp đồng** cộng **đơn giá r theo số giờ** hợp đồng bao phủ, kể cả giờ trống ở giữa. Mô hình này làm cho "ít màu nhất" chưa chắc là "rẻ nhất", nên các thuật toán cho kết quả khác nhau và có chỗ để so sánh.

Sau đó em muốn mở rộng thêm một bài toán thứ hai thực tế hơn, không chỉ có tàu mà cả **nhân lực cảng phục vụ tàu**. Từ đó demo được mở rộng thành bốn module: thuê bến, phân công đội, quỹ giờ công tăng ca (cái túi) và xếp container (đóng thùng). Ba module sau đều là các dạng bài đã học trong môn (tham lam, quy hoạch động, quay lui) nên có thể đối chiếu trực tiếp với bài giảng.

Nguồn ý tưởng: bài giảng CS112 của cô Huỳnh Thị Thanh Thương [1]. Cách ghép bốn bài toán thành một bối cảnh cảng là phần em tự thiết kế.

1.3 Ứng dụng thực tế

- **Thuê bến và xếp lịch cầu cảng:** gom các chuyến tàu vào ít hợp đồng thuê bến hơn giúp giảm phí cố định; đây là dạng bài toán xếp lịch khoảng (interval partitioning) và tô màu đồ thị khoảng [3][11].
- **Phân công đội bốc xếp, đội lạnh, đội kiểm định:** mỗi tàu cần một số kỹ năng, mỗi đội có đơn giá riêng; cần gán đúng kỹ năng, không để một đội phục vụ hai tàu cùng lúc.
- **Phân bổ quỹ giờ tăng ca có hạn:** chọn yêu cầu dịch vụ thêm nào để tổng giá trị lớn nhất trong quỹ giờ cho phép (cái túi).
- **Xếp container lên tàu:** dùng ít tàu nhất cho một tập lô hàng có khối lượng cho trước (đóng thùng).

Cả bốn bài toán đều là bài toán NP-khó ở dạng tổng quát [8], nên việc so sánh thuật toán tham lam (nhanh, không đảm bảo tối ưu) với quay lui có cắt nhánh (chậm hơn, cho kết quả tối ưu khi chạy xong) có ý nghĩa thực tế.

CHƯƠNG 2. MÔ TẢ BÀI TOÁN

Phần này phát biểu từng bài toán theo cách mô tả bài toán trong đề thi và trong các buổi học.

2.1 Bài toán 1: Thuê bến (tô màu đồ thị khoảng)

Một hãng tàu có n chuyến tàu. Tàu i cần một bến từ giờ s_i đến giờ e_i (s_i < e_i, giờ thập phân, có thể vượt 24 để biểu diễn tàu qua đêm). Hãng thuê bến theo **hợp đồng**. Mỗi hợp đồng chỉ phục vụ các tàu đôi một không chồng giờ. Chi phí một hợp đồng là F cộng r nhân với số giờ tính từ giờ đến của tàu đầu tiên đến giờ rời của tàu cuối cùng, **kể cả giờ trống ở giữa**. Ngoài ra có thể có một số cặp tàu bị ràng buộc thêm, không được ở chung hợp đồng dù không trùng giờ.

Hãy xếp mọi tàu vào các hợp đồng sao cho tổng chi phí nhỏ nhất.

2.2 Bài toán 2: Phân công đội

Có n lượt tàu và m đội. Tàu i cần một tập kỹ năng. Đội j có một tập kỹ năng và đơn giá rate_j mỗi giờ. Gán tàu i cho đội j được phép khi đội j có đủ kỹ năng mà tàu i cần; chi phí là rate_j nhân với thời gian tàu i ở cảng. Một đội không được phục vụ hai tàu trùng giờ, hoặc hai tàu thuộc một cặp bị ràng buộc thêm. Hãy gán mọi tàu cho một đội sao cho tổng chi phí nhỏ nhất, hoặc kết luận không có cách gán hợp lệ.

2.3 Bài toán 3: Quỹ giờ công (cái túi 0/1)

Có n yêu cầu dịch vụ thêm. Yêu cầu i tốn w_i giờ công (số nguyên) và mang lại giá trị v_i. Quỹ giờ công tăng ca là W. Mỗi yêu cầu chỉ được nhận hoặc không nhận (không nhận một phần). Hãy chọn tập yêu cầu có tổng giờ không vượt quá W và tổng giá trị lớn nhất.

2.4 Bài toán 4: Xếp container (đóng thùng)

Có n lô container, lô i nặng w_i tấn. Mỗi tàu chở tối đa C tấn. Mỗi lô phải nằm trọn trên một tàu. Hãy xếp mọi lô lên số tàu ít nhất.

2.5 Liên hệ giữa các module

Cả bốn module dùng chung một khung giao diện và cùng một dạng dữ liệu bước chạy, nên cách xem thuật toán từng bước giống nhau.

CHƯƠNG 3. MÔ HÌNH BÀI TOÁN

Mô hình hoá ở đây chỉ gồm đặc tả bài toán: Input, Output, ràng buộc và ví dụ. Cách giải nằm ở Chương 4.

3.1 Module 1: Thuê bến

Thành phần	Đặc tả
Input	Danh sách n tàu; mỗi tàu có (s_i, e_i) với s_i < e_i. Tham số F ≥ 0 (phí cố định mỗi hợp đồng), r ≥ 0 (đơn giá mỗi giờ). Tập P các cặp tàu (a, b) bị cấm chung hợp đồng (có thể rỗng).
Output	Một cách tô màu c: {1..n} → {1..K}, c(i) là hợp đồng của tàu i. Tổng chi phí cost(c).
Ràng buộc	Nếu hai tàu i, j chồng giờ (s_i < e_j và s_j < e_i) hoặc (i, j) ∈ P thì c(i) ≠ c(j).
Mục tiêu	Cực tiểu cost(c) = Σ_k [F + r × (max e_i − min s_i)], lấy trên các tàu i có c(i) = k.
Cận dưới	ω·F + r·Σ(e_i − s_i), trong đó ω là số tàu chồng nhau nhiều nhất tại một thời điểm.
Giới hạn demo	n ≤ 200.

Bảng 3.1. Đặc tả module 1

Nhận xét về cận dưới. Trên đồ thị khoảng, số màu tối thiểu bằng ω [3][11], và trong mỗi hợp đồng các tàu không chồng nhau nên khoảng phủ không ngắn hơn tổng thời gian các tàu. Vì vậy chi phí không thể thấp hơn ω·F + r·Σ(e_i − s_i). Cận này không phải lúc nào cũng đạt được (khi có giờ trống bắt buộc hoặc ràng buộc thêm), nên nó chỉ dùng để cắt nhánh và đánh giá mức chênh lệch.

Ví dụ 3.1. (testcase m1-T05) 8 tàu, F = 500, r = 40, cặp cấm (Tàu 8, Tàu 7) và (Tàu 7, Tàu 6).

Tàu	Đến	Rời	Tàu	Đến	Rời
Tàu 1	11:00	19:00	Tàu 5	00:30	05:00
Tàu 2	05:30	11:00	Tàu 6	00:30	04:00
Tàu 3	01:00	03:30	Tàu 7	10:00	14:30
Tàu 4	10:30	12:30	Tàu 8	03:00	08:00

Output (tối ưu): 4 hợp đồng, tổng chi phí 3780 (HĐ 1: Tàu 6, 2, 1; HĐ 2: Tàu 5, 7; HĐ 3: Tàu 3; HĐ 4: Tàu 8, 4). Cận dưới: ω = 4, Σ(e − s) = 35,5 giờ, nên cận = 4 × 500 + 40 × 35,5 = 3420.

3.2 Module 2: Phân công đội

Thành phần	Đặc tả
Input	n tàu; mỗi tàu có (s_i, e_i) và tập kỹ năng cần need_i. m đội; mỗi đội có tập kỹ năng skills_j và đơn giá rate_j. Tập các cặp tàu bị cấm chung đội (có thể rỗng).
Output	Hàm gán a: tàu → đội, kèm tổng chi phí; hoặc kết luận vô nghiệm.
Ràng buộc	need_i ⊆ skills_a(i). Hai tàu chồng giờ hoặc thuộc cặp cấm thì không cùng đội.

Thành phần	Đặc tả
Mục tiêu	Cực tiểu Σ_i rate_a(i) × (e_i − s_i).
Cận dưới	Σ_i (chi phí của đội rẻ nhất đủ kỹ năng cho tàu i).
Giới hạn demo	n ≤ 60 tàu, m ≤ 12 đội.

Bảng 3.2. Đặc tả module 2

Ví dụ 3.2. (testcase m2-T01) 6 tàu và 4 đội: Đội 1 (buộc dây, 24/giờ), Đội 2 (hóa chất, buộc dây, hàng lạnh, cẩu; 19/giờ), Đội 3 (cẩu, 32/giờ), Đội 4 (cẩu, hóa chất, hàng lạnh; 20/giờ). Tàu 1, 3 và 5 cần buộc dây; Tàu 4 cần hàng lạnh; Tàu 6 cần hóa chất; Tàu 2 không cần kỹ năng; cặp cấm (Tàu 1, Tàu 3). Output tối ưu có tổng chi phí 451,5.

Ví dụ vô nghiệm. (testcase m2-D01) ba tàu cùng neo từ giờ 0 đến giờ 5 và cùng cần cần cẩu, nhưng chỉ có hai đội đủ kỹ năng: Output là "vô nghiệm" (vi phạm điều kiện Hall, mục 4.3.3).

3.3 Module 3: Quỹ giờ công (cái túi 0/1)

Thành phần	Đặc tả
Input	n yêu cầu; mỗi yêu cầu có w_i (số nguyên ≥ 1) và v_i ≥ 0. Quỹ W (số nguyên).
Output	Tập chỉ số S ⊆ {1..n}, tổng giá trị Σ_{i∈S} v_i.
Ràng buộc	Σ_{i∈S} w_i ≤ W.
Mục tiêu	Cực đại Σ_{i∈S} v_i.
Giới hạn demo	n ≤ 25, W ≤ 200, n × (W + 1) ≤ 2500 (kích thước bảng quy hoạch động).

Bảng 3.3. Đặc tả module 3

Ví dụ 3.3. (testcase m3-T04) W = 50; A (10 giờ, 60), B (20 giờ, 100), C (30 giờ, 120). Output tối ưu: chọn {B, C}, tổng giá trị 220. Tham lam theo giá trị/giờ chọn {A, B}, chỉ được 160.

3.4 Module 4: Xếp container (đóng thùng)

Thành phần	Đặc tả
Input	n lô với khối lượng w_i > 0; tải trọng mỗi tàu C > 0, w_i ≤ C.
Output	Phân hoạch các lô thành các nhóm (tàu), kèm số tàu dùng.
Ràng buộc	Tổng khối lượng mỗi nhóm không vượt C.
Mục tiêu	Cực tiểu số nhóm.
Cận dưới	⌈Σ w_i / C⌉.
Giới hạn demo	n ≤ 60.

Bảng 3.4. Đặc tả module 4

Ví dụ 3.4. (testcase m4-T04) C = 10, các lô 2, 5, 6, 2, 5, 1, 6, 7, 2 (tổng 36, cận dưới 4). Output tối ưu 4 tàu. First-Fit theo thứ tự cho sẵn dùng 5 tàu.

CHƯƠNG 4. THIẾT KẾ CẤU TRÚC DỮ LIỆU VÀ THUẬT TOÁN

Các nguồn được đánh số [1]–[11] theo mục Tài liệu tham khảo. Phần nào do em tự thiết kế được ghi rõ ở cuối mỗi mục.

4.1 Cấu trúc dữ liệu chung và cách sinh bước

Mỗi thuật toán không chỉ trả kết quả mà trả một danh sách bước. Mỗi bước là một bản ghi gồm: loại bước (kind: info, assign, back, prune, found, done; riêng module 2 có thêm dead, module 3 có thêm skip, cell, trace), thông điệp mô tả bằng tiếng Việt (msg), trạng thái hiện tại (tô màu từng tàu, chi phí đã tiêu, phương án tốt nhất hiện có, độ sâu) và bảng ứng viên kèm lý do bị loại. Backend (FastAPI) chạy mọi thuật toán của một module trong một lần gọi (một endpoint cho mỗi module) và trả toàn bộ các bước; giao diện React duyệt các bước từ bộ nhớ, không gọi lại máy chủ.

Hai chiến lược trình bày chung: (1) mã giả được tô sáng đúng dòng ứng với loại bước đang xem; (2) với quay lui, giao diện hiển thị phương án tốt nhất hiện có và cận dưới cạnh trạng thái hiện tại. Mỗi thuật toán trả tối đa 3000 bước (ít hơn khi n lớn, vì n nhân số bước không vượt 400.000). Quay lui ở cả bốn module bắt đầu với phương án tốt nhất hiện có lấy từ kết quả tham lam tốt nhất, và dừng theo số lần thử chứ không theo đồng hồ: 60.000 lần thử ở module 1, 2, 3 và 400.000 ở module 4. Vì vậy cùng một đầu vào luôn cho cùng một kết quả; khi hết ngân sách, kết quả được đánh dấu "chưa chắc tối ưu". Thời gian chạy (ms) chỉ để hiển thị.

4.2 Module 1: Thuê bến

4.2.1 Cấu trúc dữ liệu

- **Danh sách tàu** `ships[i] = (tên, s, e)`.
- **Đồ thị xung đột** G = (V, E) dưới dạng danh sách kề (tập các đỉnh kề): cạnh (i, j) khi hai tàu chồng giờ hoặc thuộc cặp cấm. Dựng đồ thị tốn $O(n^2)$.
- **Mảng màu** `col[i]` (−1 nếu chưa tô) và với mỗi hợp đồng k: giờ đến đầu `lo[k]`, giờ rời cuối `hi[k]`, số tàu `cnt[k]` (dùng cho tham lam chọn hợp đồng rẻ nhất và quay lui). Chi phí hiện tại được cập nhật dần và hoàn tác được khi quay lui.
- **Thứ tự duyệt** `order`: các tàu sắp theo (giờ đến, giờ rời).

4.2.2 Phân tích đặc trưng và ý tưởng giải

Bài toán không có nhiều ràng buộc cấu trúc ngoài việc các tàu chồng giờ không được chung hợp đồng, nên bước đầu tiên là **mô hình hoá thành tô màu đồ thị** (mỗi tàu một đỉnh, hai tàu chồng giờ có cạnh, mỗi màu một hợp đồng) [1]. Trên đồ thị khoảng, duyệt tàu theo giờ đến và dùng hợp đồng nhỏ nhất còn dùng được cho số hợp đồng tối thiểu [3]. Tuy nhiên mục tiêu của bài toán là **chi phí** F + r × (khoảng phủ), nên ít hợp đồng chưa chắc rẻ nhất: gộp hai tàu cách xa nhau vào một hợp đồng phải trả cả giờ trống ở giữa. Từ đó có bốn hướng:

- **Tham lam theo giờ đến:** tiêu chuẩn cục bộ là hợp đồng có số nhỏ nhất còn dùng được. Nhanh, tối ưu về **số** hợp đồng khi không có ràng buộc phụ, nhưng không xét chi phí.

- **Tham lam chọn hợp đồng rẻ nhất:** với mỗi tàu, so chi phí nối vào từng hợp đồng đang có (r × số giờ kéo dài, gồm giờ trống) với chi phí mở hợp đồng mới (F + r × giờ của tàu), chọn cách rẻ hơn.
- **DSATUR:** mỗi lượt chọn tàu có nhiều hợp đồng khác nhau bị chặn nhất (độ bão hoà lớn nhất), hoà thì chọn tàu nhiều xung đột nhất [2]. Tối ưu số màu trên nhiều lớp đồ thị, nhưng không xét chi phí.
- **Quay lui có cắt nhánh:** duyệt tàu theo giờ đến, bắt đầu với phương án tốt nhất hiện có lấy từ kết quả tham lam tốt nhất; với mỗi tàu thử lần lượt các lựa chọn (nối vào hợp đồng đang có hoặc mở mới) từ rẻ đến đắt; bỏ nhánh khi cận không thấp hơn phương án tốt nhất hiện có.

4.2.3 Thuật toán

Mã giả tham lam theo giờ đến (theo khung GREEDY trong bài giảng [1]):

```
GreedyColoring(ships, G):
  sắp ships theo giờ đến tăng dần
  for mỗi tàu v theo thứ tự:
    forb = {col[u] : u kề v, u đã tô}
    k = 1; while k ∈ forb do k = k + 1
    col[v] = k                  // hợp đồng nhỏ nhất còn dùng được
  return col
```

Mã giả DSATUR:

```
DSATUR(G):
  for lần = 1 to n do:
    v = tàu chưa tô có |forb(v)| lớn nhất (hoà: deg(v) lớn nhất, rồi chỉ số
nhỏ)
    k = hợp đồng nhỏ nhất không thuộc forb(v)
    col[v] = k
  return col
```

Mã giả quay lui có cắt nhánh (theo khung try(i) của bài giảng [1]):

```
best = chi phí của kết quả tham lam tốt nhất; bestCol = kết quả đó
Try(i, cost):                        // i: tàu thứ i theo giờ đến
   if i = n then:                    // đã xếp hết tàu
      if cost < best then best = cost, bestCol = col
      return
   for mỗi lựa chọn k của tàu i, từ rẻ đến đắt:
      // k: nối vào hợp đồng đang có (không bị chặn) hoặc mở hợp đồng mới
      d = chi phí tăng thêm của lựa chọn k
      gán tàu i vào k;  used' = số hợp đồng đang mở
      bound = cost + d + F × max(0, ω − used')
      if bound < best then Try(i + 1, cost + d)   // ngược lại: bỏ nhánh
      bỏ gán tàu i                                // lùi lại
```

Cận cắt nhánh. Chi phí chỉ tăng khi thêm tàu, nên chi phí tạm là cận dưới của chi phí cuối. Ngoài ra số hợp đồng cuối không thể ít hơn ω (ω tàu chồng nhau phải nằm ở ω hợp đồng khác nhau), nên nếu hiện mới mở used' hợp đồng thì còn phải trả thêm ít nhất F × (ω

− used'). Cận = chi phí tạm + F × max(0, ω − used') không đánh giá cao hơn chi phí thực nên cắt nhánh không bỏ mất nghiệm tối ưu. Cận này chưa cộng phần r × thời gian của các tàu chưa xếp (bản đầu tiên có phần này), nên cắt nhánh yếu hơn. Nếu tìm được phương án có chi phí bằng cận dưới toàn cục ω·F + r·Σ(e − s) thì dừng ngay vì chắc chắn tối ưu.

4.2.4 Độ phức tạp

Thuật toán	Thời gian	Nhận xét
Dựng đồ thị	$O(n^2)$	Xét mọi cặp tàu
Tham lam theo giờ đến	O(n log n + n·k + E)	k: số hợp đồng, E: số cạnh xung đột. Không đảm bảo tối ưu về chi phí
Tham lam hợp đồng rẻ nhất	O(n log n + n·k + E)	Cũng không đảm bảo tối ưu
DSATUR	O(n·(n + E))	Chọn đỉnh bằng duyệt tuần tự
Quay lui có cắt nhánh	$O(k^n)$ tệ nhất	Số nút không vượt số cách phân hoạch n tàu (số Bell); cắt nhánh làm giảm đáng kể trong thực tế

Bảng 4.1. Độ phức tạp module 1

Bản cài đặt trong demo tốn thêm chi phí so với các công thức trên vì mỗi bước còn dựng bảng ứng viên và tính chi phí tăng thêm cho giao diện; các công thức là của thuật toán, không phải thời gian chạy đo được.

Phần tự thiết kế: mô hình chi phí hợp đồng, cận dưới ω·F + r·Σ(e − s), cận cắt nhánh F × max(0, ω − used'), thứ tự thử lựa chọn từ rẻ đến đắt, việc khởi tạo quay lui từ kết quả tham lam, thuật toán tham lam chọn hợp đồng rẻ nhất.

4.3 Module 2: Phân công đội

4.3.1 Cấu trúc dữ liệu

- Danh sách đội đủ kỹ năng `elig[i]` cho mỗi tàu và thời gian cập `dur[i] = e_i − s_i`; chi phí giao tàu i cho đội j là `rate_j × dur[i]`.
- Tập xung đột `adj[i]`: các tàu chồng giờ với tàu i hoặc thuộc cặp cấm với tàu i.
- Cận theo tàu `minrate[i]` = chi phí nhỏ nhất trong các đội đủ kỹ năng; `suffix[i]` = tổng `minrate` của các tàu từ vị trí i trở đi theo thứ tự duyệt, dùng làm cận.

4.3.2 Phân tích đặc trưng và ý tưởng giải

Mỗi tàu chỉ gán được cho các đội đủ kỹ năng (danh sách cho phép), và hai tàu xung đột không cùng đội. Nếu bỏ ràng buộc kỹ năng thì bài toán là tô màu đồ thị khoảng như module 1; ràng buộc kỹ năng khiến thứ tự xử lý quan trọng: tàu ít lựa chọn nên được gán trước kẻo bị đội rẻ nhất bị chiếm mất. Từ đó có ba thuật toán:

- **Tham lam theo giờ đến:** mỗi tàu chọn đội rẻ nhất còn rảnh và đủ kỹ năng.
- **Tham lam tàu khó trước:** sắp tàu theo số đội hợp lệ tăng dần (ít lựa chọn nhất trước), rồi chọn đội rẻ nhất; có thể xử lý được những tàu mà tham lam theo giờ đến bị "trượt nghiệm".
- **Quay lui có cắt nhánh** với cận chi phí rẻ nhất của các tàu còn lại: duyệt tàu theo giờ đến (ghi chú trong mã cho biết thứ tự này tốt hơn "tàu khó trước" trên bài lớn), thử các đội đủ kỹ năng từ rẻ đến đắt, bỏ qua đội giống hệt nhau chưa có việc, và bắt

đầu từ phương án tham lam tốt nhất. Bỏ nhánh khi chi phí hiện tại cộng cận của các tàu còn lại không thấp hơn phương án tốt nhất.

4.3.3 Phát hiện vô nghiệm bằng định lý Hall

Quay lui trên dữ liệu vô nghiệm có thể phải duyệt rất lâu mới kết luận. Vì vậy trước khi tìm kiếm, thuật toán kiểm tra một **điều kiện cần**: tại mỗi thời điểm bắt đầu của một tàu, tập các tàu đang ở cảng phải gán được cho các đội khác nhau, tức tồn tại một **cặp ghép hai phía** từ các tàu này vào các đội hợp lệ. Cặp ghép được tìm bằng thuật toán đường tăng của Kuhn [10]; nếu không ghép hết được thì theo định lý Hall [9] bài toán vô nghiệm, và demo báo ngay cùng danh sách các tàu gây ra mâu thuẫn. Đây chỉ là điều kiện cần (chưa xét cặp cấm thêm), nên qua được kiểm tra chưa chắc có nghiệm; khi đó tìm kiếm tiếp tục như bình thường.

4.3.4 Độ phức tạp

Thuật toán	Thời gian	Nhận xét
Tham lam (hai kiểu)	O(n log n + n·m·d)	d: số xung đột trung bình. Có thể không gán hết được dù có nghiệm
Kiểm tra Hall	O(n·(n + m)·m) mỗi mốc giờ	Ghép hai phía theo Kuhn; điều kiện cần cho tính khả thi
Quay lui có cắt nhánh	$O(m^n)$ tệ nhất	Mỗi tàu thử tối đa m đội; cắt nhánh bằng suffix

Bảng 4.2. Độ phức tạp module 2

Nguồn và phần tự thiết kế: khung tham lam, quay lui theo bài giảng [1]; kiểm tra vô nghiệm theo Hall [9] và Kuhn [10]. Mô hình bài toán, cận dưới Σ minc[i], thứ tự "tàu khó trước" và cách áp dụng ghép hai phía cho các tàu cùng cập một lúc là phần tự thiết kế.

4.4 Module 3: Quỹ giờ công (cái túi 0/1)

4.4.1 Cấu trúc dữ liệu

- Bảng quy hoạch động dp[i][w]: giá trị lớn nhất khi xét i yêu cầu đầu tiên với quỹ w giờ. Kích thước (n + 1) × (W + 1); demo giới hạn n × (W + 1) ≤ 2500 để hiển thị được (mỗi bước điền một ô).
- **Mảng chọn** take[i] dùng để truy vết và để quay lui ghi nhận nhánh đang xét.

4.4.2 Phân tích đặc trưng và ý tưởng giải

Bài toán có **cấu trúc con tối ưu**: lời giải tối ưu cho i yêu cầu, quỹ w gồm lời giải tối ưu của i − 1 yêu cầu với quỹ w hoặc w − w_i. Đây là dạng quy hoạch động như bài giảng [1] [3]. Tham lam theo giá trị/giờ giảm dần chỉ đúng với cái túi **phân số** [4]; với 0/1 thì có thể sai (ví dụ 4.1). Quay lui dùng cận trên bằng **nới lỏng phân số** [5].

Thuật toán	Ý tưởng
Tham lam	Sắp theo v/w giảm dần, nhận yêu cầu nào còn vừa quỹ.
Quy hoạch động	dp[i][w] = max(dp[i−1][w], dp[i−1][w − w_i] + v_i) nếu w ≥ w_i; truy vết để lấy tập chọn.

Thuật toán	Ý tưởng
Quay lui + cận	Duyệt yêu cầu theo giá trị/giờ giảm dần; mỗi yêu cầu: nhận hoặc bỏ; cận trên = giá trị hiện có + nới lỏng phân số phần còn lại; bỏ nhánh khi cận ≤ phương án tốt nhất hiện có (bắt đầu từ kết quả tham lam).

Bảng 4.3. Ba thuật toán của module 3

Ví dụ 4.1. (testcase m3-T04, đã nêu ở Ví dụ 3.3) Tham lam theo v/w nhận A (6/giờ) rồi B (5/giờ), hết chỗ cho C: tổng 160. Tối ưu là nhận B và C: 220. Ở testcase lớn m3-L01 (25 yêu cầu, quỹ 100) tham lam cũng thấp hơn: 970 so với 997.

4.4.3 Độ phức tạp

Thuật toán	Thời gian	Bộ nhớ
Tham lam	O(n log n)	O(n)
Quy hoạch động	O(n·W)	O(n·W) (cần bảng để truy vết)
Quay lui + cận	Tệ nhất O(2^n)	O(n) ngăn xếp

Bảng 4.4. Độ phức tạp module 3

Quy hoạch động có thời gian giả đa thức (phụ thuộc giá trị W chứ không chỉ kích thước dữ liệu), nên với W lớn không dùng được; demo giới hạn W ≤ 200.

Phần tự thiết kế: bối cảnh "quỹ giờ công tăng ca" và các thông điệp từng bước.

4.5 Module 4: Xếp container (đóng thùng)

4.5.1 Cấu trúc dữ liệu

- **Mảng gán** `bin[i]` (lô i nằm trên tàu nào) và mảng tải `loads[b]` = tổng khối lượng đã xếp lên tàu b.
- **Cận dưới** ⌈Σ w_i / C⌉ [6].

4.5.2 Phân tích đặc trưng và ý tưởng giải

Số tàu không thể ít hơn ⌈Σ w_i / C⌉. Các heuristic xếp từng lô vào tàu có sẵn [7]:

- **First-Fit (FF):** xếp lô vào tàu đầu tiên còn đủ chỗ, theo thứ tự cho sẵn.
- **First-Fit Decreasing (FFD):** sắp lô giảm dần theo khối lượng rồi First-Fit.
- **Best-Fit Decreasing (BFD):** sắp giảm dần, xếp vào tàu vừa khít nhất (chỗ còn lại nhỏ nhất).
- **Quay lui có cắt nhánh:** bắt đầu với phương án tốt nhất của First-Fit, First-Fit giảm dần và Best-Fit giảm dần; xếp lần lượt các lô nặng trước, mỗi lô thử vào các tàu đang mở đủ chỗ (bỏ qua các tàu có cùng mức tải đã thử, vì đối xứng) rồi thử mở tàu mới; bỏ nhánh khi cận (số tàu hiện có + ⌈(khối lượng chưa xếp − chỗ trống đang có) / C⌉) không thấp hơn phương án tốt nhất; dừng ngay khi chạm cận dưới ⌈Σ w / C⌉.

Sắp giảm dần có tác dụng đặt các lô lớn trước, khó xếp nhất, nên FFD và BFD thường tốt hơn FF (ví dụ 3.4: FF dùng 5 tàu, FFD dùng 4).

4.5.3 Độ phức tạp

Thuật toán	Thời gian	Chất lượng
FF	$O(n^2)$ (duyệt các tàu có sẵn)	Không quá khoảng 1,7 lần tối ưu [7]
FFD, BFD	$O(n \log n + n^2)$	Không quá khoảng 11/9 lần tối ưu cộng hằng số [7]
Quay lui + cận	Tệ nhất mũ	Tối ưu nếu chạy xong

Bảng 4.5. Độ phức tạp và chất lượng module 4

Nguồn và phần tự thiết kế: FF, FFD, BFD theo Johnson và cộng sự [7]; cận dưới theo Martello và Toth [6]; khung quay lui theo bài giảng [1]. Cách cài đặt bỏ qua tàu cùng tải, các thông điệp từng bước là phần tự thiết kế.

4.6 Tính đúng đắn và cách đánh giá chất lượng

- Tham lam không đảm bảo tối ưu ở cả bốn module: Ví dụ 3.1 (module 1: tham lam 3960 so với tối ưu 3780), testcase m2-D02 (module 2: tham lam theo giờ đến không gán được dù có nghiệm), Ví dụ 4.1 (module 3) và Ví dụ 3.4 (module 4). Đây là lý do cần các thuật toán so sánh.
- **Quay lui có cắt nhánh** cho kết quả tối ưu khi duyệt xong (nhãn **"TỐI ƯU"** trong demo); nếu hết ngân sách số lần thử (60.000 ở module 1, 2, 3; 400.000 ở module 4) thì kết quả được đánh dấu "CHƯA CHẮC TỐI ƯU".
- **Quy hoạch động** (module 3) cho kết quả tối ưu vì đúng cấu trúc con tối ưu.
- Chất lượng mỗi kết quả được so với **cận dưới (hoặc cận trên)** ở trên để biết còn cách tối ưu bao xa.

CHƯƠNG 5. VÍ DỤ MINH HOẠ CÁC BƯỚC THỰC HIỆN THUẬT TOÁN

Chương này trình bày chi tiết một ví dụ của module 1 (testcase m1-T05, đã nêu ở Ví dụ 3.1). Ví dụ làm lộ rõ khác biệt giữa bốn thuật toán: **cả bốn đều dùng 4 hợp đồng nhưng cho bốn tổng chi phí khác nhau**. Các bước và thông điệp dưới đây lấy từ chính danh sách bước mà demo trả về, nên khớp với phần phát từng bước trên giao diện: nhập dữ liệu của testcase m1-T05 vào giao diện module 1 (F = 500, r = 40, hai cặp cấm (Tàu 8, Tàu 7) và (Tàu 7, Tàu 6)) rồi chọn từng thuật toán.

5.1 Dữ liệu và đồ thị xung đột

Thứ tự các tàu theo giờ đến: Tàu 6, Tàu 5, Tàu 3, Tàu 8, Tàu 2, Tàu 7, Tàu 4, Tàu 1. Đồ thị xung đột có 14 cạnh: (1,4), (1,7), (2,4), (2,7), (2,8), (3,5), (3,6), (3,8), (4,7), (5,6), (5,8), (6,7), (6,8), (7,8). Hai cạnh (6,7) và (7,8) đến từ ràng buộc thêm (hai tàu không chồng giờ). Bậc các đỉnh: Tàu 1: 2; Tàu 2: 3; Tàu 3: 3; Tàu 4: 3; Tàu 5: 3; Tàu 6: 4; Tàu 7: 5; Tàu 8: 5. Số tàu chồng nhau nhiều nhất ω = 4 (từ giờ 3 đến giờ 3,5 có bốn tàu cùng neo: Tàu 3, 5, 6, 8), tổng thời gian các tàu bằng 35,5 giờ, nên cận dưới là 4 × 500 + 40 × 35,5 = 3420.

5.2 Tham lam theo giờ đến

Mỗi tàu được xếp vào hợp đồng có số nhỏ nhất không bị chặn, không xét chi phí.

Bước	Tàu xét (đến–rời)	Hợp đồng bị chặn	Chọn	Tăng thêm	Tổng
1	Tàu 6 (00:30–04:00)	không có	HĐ 1	640	640
2	Tàu 5 (00:30–05:00)	HĐ 1	HĐ 2	680	1320
3	Tàu 3 (01:00–03:30)	HĐ 1, 2	HĐ 3	600	1920
4	Tàu 8 (03:00–08:00)	HĐ 1, 2, 3	HĐ 4	700	2620
5	Tàu 2 (05:30–11:00)	HĐ 4	HĐ 1	280	2900
6	Tàu 7 (10:00–14:30)	HĐ 1, 4	HĐ 2	380	3280
7	Tàu 4 (10:30–12:30)	HĐ 1, 2	HĐ 3	360	3640
8	Tàu 1 (11:00–19:00)	HĐ 2, 3	HĐ 1	320	3960

Bảng 5.1. Tham lam theo giờ đến: 4 hợp đồng, chi phí 3960

Tàu 2 vào HĐ 1 vì HĐ 1 là số nhỏ nhất còn dùng được, dù nối vào HĐ 1 phải trả cả 1,5 giờ trống giữa Tàu 6 (rời 04:00) và Tàu 2 (đến 05:30). Tàu 7 vào HĐ 2 phải trả 5 giờ trống và Tàu 4 vào HĐ 3 phải trả 7 giờ trống. Kết quả dùng đúng ω = 4 hợp đồng nhưng tốn 3960, cao hơn tối ưu 180.

5.3 Tham lam chọn hợp đồng rẻ nhất

Với mỗi tàu, so chi phí nối vào hợp đồng đang có (kể cả giờ trống) với chi phí mở mới F + r × giờ của tàu.

Bước	Tàu xét	Quyết định	Chi phí	Tổng
1	Tàu 6	Chưa có hợp đồng, mở HĐ 1	500 + 40×3,5 = 640	640

Bước	Tàu xét	Quyết định	Chi phí	Tổng
2	Tàu 5	Bị chặn bởi HĐ 1; mở HĐ 2	500 + 40×4,5 = 680	1320
3	Tàu 3	Bị chặn bởi HĐ 1, 2; mở HĐ 3	500 + 40×2,5 = 600	1920
4	Tàu 8	Bị chặn bởi HĐ 1, 2, 3; mở HĐ 4	500 + 40×5 = 700	2620
5	Tàu 2	Nối HĐ 2 (rảnh từ 05:00, trống 0,5 giờ)	40×6 = 240 < mở mới 720	2860
6	Tàu 7	Nối HĐ 3 (rảnh từ 03:30, trống 6,5 giờ)	40×11 = 440 < mở mới 680	3300
7	Tàu 4	Nối HĐ 4 (rảnh từ 08:00, trống 2,5 giờ)	40×4,5 = 180 < mở mới 580	3480
8	Tàu 1	Nối HĐ 2 (rảnh từ 11:00, trống 0 giờ)	40×8 = 320 < mở mới 820	3800

Bảng 5.2. Tham lam hợp đồng rẻ nhất: 4 hợp đồng, chi phí 3800

5.4 DSATUR

Mỗi lượt chọn tàu có nhiều hợp đồng bị chặn nhất; hoà thì chọn tàu nhiều xung đột nhất.

Bước	Tàu chọn	Bị chặn / xung đột	Hợp đồng	Tăng thêm
1	Tàu 7	0 hợp đồng / 5 xung đột	HĐ 1	680
2	Tàu 8	1 (HĐ 1) / 5	HĐ 2	700
3	Tàu 6	2 (HĐ 1, 2) / 4	HĐ 3	640
4	Tàu 2	2 (HĐ 1, 2) / 3	HĐ 3	280
5	Tàu 3	2 (HĐ 2, 3) / 3	HĐ 1	360
6	Tàu 5	3 (HĐ 1, 2, 3) / 3	HĐ 4	680
7	Tàu 4	2 (HĐ 1, 3) / 3	HĐ 2	180
8	Tàu 1	2 (HĐ 1, 2) / 2	HĐ 3	320

Bảng 5.3. DSATUR: 4 hợp đồng, chi phí 3840

DSATUR dùng đúng ω = 4 hợp đồng nhưng tổng chi phí 3840 vẫn cao hơn tối ưu 3780, vì thuật toán không xét giờ trống giữa các tàu cùng hợp đồng.

5.5 Quay lui có cắt nhánh

Quay lui bắt đầu với phương án tốt nhất hiện có là kết quả tham lam rẻ nhất (3800) và trả về 108 bước (27 bước thử, 27 bước quay lui, 1 lần tìm được phương án tốt hơn, 51 lần bỏ nhánh, cộng bước mở đầu và bước kết thúc), tương ứng 78 lần thử gán một tàu. Bảng dưới tóm tắt các giai đoạn.

Giai đoạn	Diễn biến	Phương án tốt nhất
1. Đi xuống sâu nhất	Thử lần lượt Tàu 6 → HĐ 1 (640), Tàu 5 → HĐ 2 (1320), Tàu 3 → HĐ 3 (1920), Tàu 8 → HĐ 4 (2620), Tàu 2 → HĐ 2 (2860), Tàu 7 → HĐ 3 (3300), Tàu 4 → HĐ 4 (3480). Với Tàu 1, mọi lựa chọn đều cho cận không thấp hơn 3800 (HĐ 2: 3800; HĐ 1: 4080; mở mới: 4300): bỏ nhánh cả ba.	3800 (từ tham lam)
2. Lùi và thử lại ở các tàu cuối	Lùi tới Tàu 4: thử HĐ 1 (3640) nhưng Tàu 1 vẫn bị bỏ nhánh; mở hợp đồng mới cho Tàu 4 cho cận 3880 ≥ 3800: bỏ nhánh. Lùi tới Tàu 7 thử mở hợp đồng mới (3540), rồi các lựa chọn của Tàu 4 và Tàu 1 phía dưới đều bị bỏ nhánh.	3800
3. Nhánh mới tìm được phương án rẻ hơn	Lùi tới Tàu 2, thử HĐ 1 (2900); Tàu 7 → HĐ 2 (3280); Tàu 4 → HĐ 4 (3460); Tàu 1 → HĐ 1 (3780). Đã xếp hết: tìm được phương án 4 hợp đồng, 3780 < 3800.	3780
4. Duyệt nốt	Các nhánh còn lại của Tàu 2, 7, 4 và 1 đều bị bỏ nhánh vì cận không thấp hơn 3780. Tàu 8, 3, 5, 6 không còn lựa chọn khác, nên lùi hết về Tàu 6 và duyệt xong toàn bộ cây tìm kiếm.	3780 (tối ưu)

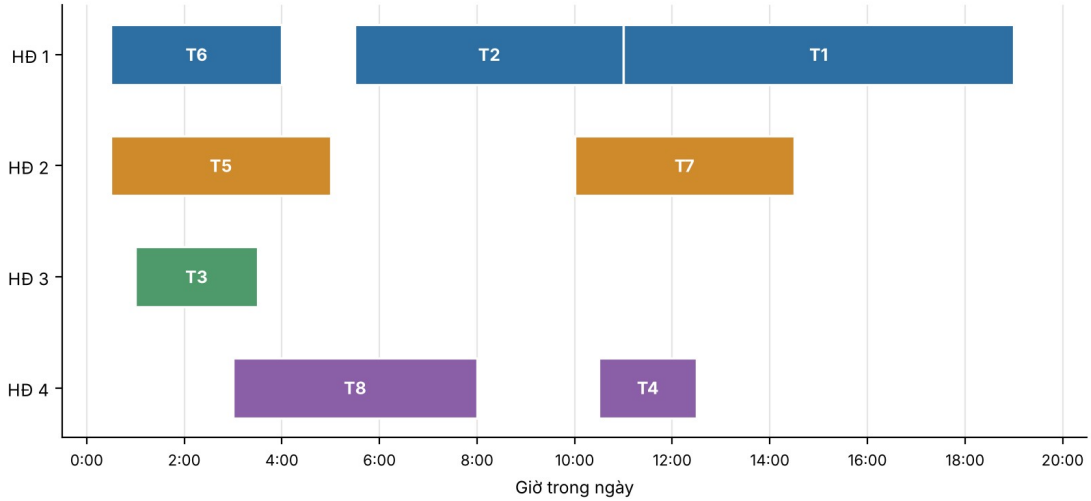
Bảng 5.4. Các giai đoạn của quay lui

Cận dưới toàn cục là 3420, thấp hơn 3780, nên thuật toán không dừng sớm mà duyệt hết cây; vì duyệt xong (78 lần thử, không chạm ngân sách 60.000) nên kết quả mang nhãn "TỐI ƯU". Kết quả khớp với đáp án vét cạn của testcase (3780).

5.6 Kết quả tổng hợp

Thuật toán	Số hợp đồng	Tổng chi phí	So với tối ưu 3780
Tham lam theo giờ đến	4	3960	+180 (+4,8%)
Tham lam hợp đồng rẻ nhất	4	3800	+20 (+0,5%)
DSATUR	4	3840	+60 (+1,6%)
Quay lui có cắt nhánh	4	**3780**	0

Bảng 5.5. Kết quả bốn thuật toán trên ví dụ m1-T05



Hình 5.1. Lời giải tối ưu (3780): HĐ 1 gồm Tàu 6, 2, 1; HĐ 2 gồm Tàu 5, 7; HĐ 3 gồm Tàu 3; HĐ 4 gồm Tàu 8, 4. Chi phí từng hợp đồng: 1240, 1060, 600, 880.

Tổng bốn hợp đồng: (500 + 40×18,5) + (500 + 40×14) + (500 + 40×2,5) + (500 + 40×9,5) = 1240 + 1060 + 600 + 880 = 3780.

CHƯƠNG 6. TƯỜNG THUẬT QUÁ TRÌNH THIẾT KẾ VÀ XÂY DỰNG

Chương này kể lại hai giai đoạn. Mục 6.1 đến 6.6 kể lại quá trình thật của bản đầu tiên (10–14/10/2026), nay giữ làm bản tham chiếu. Mục 6.7 kể lại bản dựng lại bằng AI, là bản em nộp, và nói rõ ai làm gì.

6.1 Công cụ và phạm vi sử dụng AI

Em dùng **Claude (Anthropic)** làm trợ lý lập trình, qua giao diện trò chuyện và qua công cụ lập trình trên môi trường đám mây. Ở bản đầu tiên em không dùng một framework phát triển AI nào (như Spec Kit hay Superpowers) trong quá trình làm (mục 6.6); bản dựng lại ở mục 6.7 thì đi theo các bước kiểu Spec Kit.

Công đoạn	AI làm gì	Em làm gì
Chọn đề tài, mô hình	Giải thích và gợi ý cách mô hình hoá khi em hỏi	Chọn đề tài từ bài toán Sở thú, đổi sang góc nhìn hãng tàu, quyết định mô hình chi phí F + r × giờ và mở rộng lên bốn module
Thiết kế kiến trúc	Đề xuất tách backend/giao diện, định dạng danh sách bước	Chọn FastAPI cho backend, chọn Codespaces để chạy demo, muốn tự gõ lệnh cài đặt và chạy (nên dùng npm scripts)
Cài đặt	Viết nháp mã nguồn thuật toán, sinh bước, giao diện	Duyệt, giữ hay bỏ, yêu cầu sửa
Kiểm thử	Sinh bộ testcase, bộ chạy độc lập, kiểm tra vô nghiệm	Giới hạn phạm vi: chỉ kiểm hàm thuật toán; yêu cầu bỏ phần kiểm lớp API
Dọn dẹp, tài liệu	Gắn nguồn vào mã, viết README, nháp tài liệu	Đối chiếu với đề của cô, hỏi lại nguyên nhân khi phát hiện lỗi, quyết định tự kiểm chứng nguồn sau

Bảng 6.1. Phân vai giữa AI và em

6.2 Dòng thời gian

Mốc	Nội dung	Bằng chứng (git)
Trước 10/10	Chọn đề tài (Sở thú → thuê bến), chọn mô hình chi phí, chọn công nghệ và cách chạy	Trao đổi trong cuộc trò chuyện riêng
10/10/2026	Bản đầu của module 1: backend FastAPI, giao diện React, devcontainer; bỏ script tự chạy, chuyển sang npm scripts; thêm tab "Từng bước" (mã giả, bảng ứng viên, biểu đồ cận)	4 commit: 63a6cfd, be15e4b, 5ff13b4, e21de19
12/10/2026	Mở rộng thêm 3 module (phân công đội, quỹ giờ công, xếp container); sinh bộ testcase; chỉnh phạm vi kiểm thử; dọn kho mã; gắn nguồn tham khảo	Các commit c8cdecd, 149fc94, 3a5da29, 354cc4f, 9c39abd, e82fa6f, cd5415a
13–14/10/2026	Tài liệu prompt dựng lại, nhật ký sử dụng AI, bản nháp report này	Phụ lục B, C
15/10/2026	Dựng lại bốn module vào thư mục replication/app theo một đặc tả viết trước; kiểm bằng 79 testcase và 237 bài pytest; đối chiếu với bản đầu tiên (mục 6.7)	Các commit 19a6f3e, 4a104d8, e8b8dec, 116c1c6

Bảng 6.2. Các giai đoạn thực hiện

6.3 Các prompt đã sử dụng

Phụ lục B liệt kê các tin nhắn em đã gửi trong phiên làm việc ngày 12–13/10, gồm bản gốc tiếng Việt và bản tiếng Anh đã chỉnh cho rõ ràng, kèm việc AI đã làm và quyết định của em. Các tin nhắn em viết ngắn, theo kiểu trao đổi nhanh, và thường là yêu cầu điều chỉnh sau khi xem kết quả. Vài ví dụ tiêu biểu:

- **"viết test case giúp mình"** → AI sinh bộ 122 testcase, gồm cả nhóm kiểm lớp API.
- **"không test case ở đâu là test cái hàm thể hiện thuật toán thôi"** → em chỉnh lại phạm vi; AI bỏ nhóm API/dữ liệu không hợp lệ, bộ chạy gọi thẳng hàm giải, còn 88 testcase.
- **"`e2e.png` đang nằm trong repo, nên xoá"** rồi **"vì sao testcase lại sinh ra ảnh để phải xoá"** → AI xoá ảnh, thêm vào .gitignore, sửa script ghi ảnh vào thư mục tạm và giải thích đó là lỗi của AI khi dùng lệnh git add -A.

Các tin nhắn của ngày 10/10 (xây bản đầu module 1) không còn lưu lại nguyên văn; bằng chứng còn lại là bốn commit ở Bảng 6.2.

6.4 Vấn đề gặp phải và cách giải quyết

Vấn đề	Nguyên nhân	Cách giải quyết
Testcase lớn của module 2 vô tình vô nghiệm, quay lui không chứng minh được vô nghiệm trong thời gian cho phép	Dữ liệu sinh ngẫu nhiên vô tình có thời điểm cần nhiều đội hơn số đội hiện có; quay lui phải duyệt rất lâu mới kết luận	Thêm hàm kiểm tra vô nghiệm bằng định lý Hall và ghép hai phía (mục 4.3.3); sinh lại testcase, thêm một ca lớn khả thi (m2-L01) và một ca lớn vô nghiệm có chủ đích (m2-L04)
Mô tả của vài testcase không khớp với kết quả chạy thật (ví dụ ghi "tham lam kém tối ưu" nhưng kết quả bằng tối ưu)	Mô tả được viết theo dự đoán trước khi chạy	Chạy thật, đối chiếu và sửa lại dữ liệu hoặc mô tả theo kết quả đo
Bộ testcase ban đầu có nhóm kiểm lớp API (mã lỗi 422)	AI hiểu phạm vi kiểm thử rộng hơn nhu cầu	Em yêu cầu chỉ kiểm hàm thuật toán. Bỏ nhóm đó, 122 ca còn 88; bộ chạy gọi thẳng hàm giải
Ảnh chụp thử e2e.png bị đưa vào kho mã	Lệnh git add -A của AI gom cả ảnh do script kiểm thử giao diện sinh ra	Xoá khỏi kho, thêm vào .gitignore, sửa script ghi ảnh vào thư mục tạm
Nhãn "TỐI ƯU" hiện sai ở ca vô nghiệm	Giao diện chưa phân biệt vô nghiệm với tối ưu	Nhãn mới: "VÔ NGHIỆM", "TỐI ƯU", "CHƯA CHẮC TỐI ƯU"; chạy lại kiểm thử giao diện
Kết quả quay lui ở các ca lớn phụ thuộc vào giới hạn thời gian 1,5 giây	Quay lui dừng theo đồng hồ chứ không theo số nút	Chưa sửa ở bản đầu tiên. Bản nộp (mục 6.7) dừng theo số lần thử nên kết quả lặp lại được
Ban đầu có script tự cài và tự chạy khi mở Codespaces	Bản đầu cấu hình tự cài đặt và tự chạy khi mở môi trường	Em muốn tự gõ lệnh; bỏ script tự chạy, dùng npm scripts (npm run setup, npm run dev)

Bảng 6.3. Các vấn đề và cách xử lý

6.5 Kiểm chứng, chỉnh sửa và phát triển tiếp kết quả

Kiểm chứng tự động. Mã nguồn thuật toán của bản nộp được kiểm bằng 237 bài kiểm thử pytest, trong đó có so sánh với vét cạn trên dữ liệu nhỏ của cả bốn module và toàn bộ 79 testcase (bản đầu tiên có 102 bài pytest và 88 testcase). Bộ chạy testcase là một bộ kiểm tra độc lập: nó không tin giá trị chi phí do thuật toán báo, mà tự tính lại chi phí và tự kiểm tra tính hợp lệ của lời giải, rồi so với đáp án tham chiếu lấy từ vét cạn, công thức, chương trình quy hoạch động viết riêng hoặc instance dựng sẵn (không lấy từ chính thuật toán). Giao diện được kiểm bằng kịch bản Playwright cho cả bốn module.

Quyết định và chỉnh sửa của em (ở bản đầu tiên). Em chọn và đổi hướng đề tài; chọn mô hình chi phí; giới hạn phạm vi kiểm thử về hàm thuật toán; chất vấn lỗi đưa ảnh vào kho mã thay vì chấp nhận ngay; đối chiếu dự án với từng yêu cầu của cô để tìm phần còn thiếu (bộ testcase, nguồn tham khảo, nhật ký prompt); và quyết định kiểm chứng lại các nguồn tham khảo bằng tay.

[Em bổ sung ở đây: những bước em tự kiểm tra bằng tay, ví dụ tự lần theo một lượt chạy trên giấy rồi đối chiếu với tab "Từng bước", hoặc tự đọc lại mã nguồn của một module.]

Phát triển tiếp. Sau khi có bản đầu cho module 1, em mở rộng thành bốn module, thêm bảng so sánh thuật toán, kiểm tra vô nghiệm và bộ testcase có đáp án tham chiếu độc lập.

6.6 Tài liệu prompt dựng lại theo framework (bổ trợ)

Ngoài quá trình thật ở trên, em soạn thêm một tài liệu bổ trợ gồm 10 prompt được **dựng lại sau khi dự án hoàn thành**, tổ chức theo quy trình của GitHub Spec Kit [12] (so sánh với Superpowers [13]). Tài liệu nói rõ đây là ví dụ dựng lại, **không phải prompt lịch sử**, và dự án này không dùng hay cài Spec Kit hay Superpowers. Tài liệu chỉ nhằm cho thấy cách các giai đoạn phân tích yêu cầu, khảo sát mã, thiết kế, cài đặt và kiểm thử có thể được dẫn dắt bằng prompt có cấu trúc, và đánh giá điểm mạnh, điểm yếu của từng prompt. Tài liệu đính kèm là Phụ lục C.

6.7 Bản dựng lại bằng AI (bản nộp)

Bản em nộp là bản dựng lại, nằm trong thư mục replication/app của kho mã. Sau khi bản đầu tiên (mục 6.1 đến 6.5) xong, em nhờ Claude dựng lại toàn bộ bốn module từ một đặc tả viết trước (replication/spec), đi theo các bước kiểu Spec Kit: nguyên tắc chung, đặc tả, làm rõ, kế hoạch, danh sách việc, cài đặt, đối chiếu. Sau đó mới đối chiếu với bản đầu tiên và em chọn nộp bản dựng lại. Bản đầu tiên vẫn nằm trong kho mã (thư mục gốc) làm bản tham chiếu.

Ai làm gì được ghi tách bạch trong thư mục replication/. Các prompt em thật sự gõ cho lần dựng lại này nằm nguyên văn trong replication/PROMPTS.md. Toàn bộ mã nguồn, kiểm thử và tài liệu do Claude viết. Kịch bản từng vòng trong replication/SCRIPT.md do Claude soạn sau khi dựng xong, như một đề xuất để chạy lại; trong đó mỗi quyết định có nhãn "gốc" (chấm xanh lá) nếu là quyết định thật của em ở bản đầu tiên, hoặc nhãn "đề xuất" (chấm xanh dương) nếu là đề xuất của Claude. Kịch bản không phải nhật ký những gì em đã gõ, nên không dùng làm bằng chứng cho các quyết định mang nhãn "đề xuất".

Kết quả đo được trên bản nộp: 79/79 testcase đạt (testcases/RESULTS.md), 237 bài pytest đạt, kịch bản Playwright cho cả bốn module đạt. Đối chiếu chéo với bản đầu tiên (CROSS_CHECK.md): 88 testcase của bản đầu tiên chạy trên bộ giải của bản nộp đều

đạt; 79 testcase của bản nộp chạy trên bộ giải của bản đầu tiên cho cùng kết quả tốt nhất ở 78 ca, ca còn lại (m2-L02, 60 tàu) bản nộp ra 2467,5 so với 2471,5. Trên ca m4-L02 của bản đầu tiên (60 lô, tối ưu 15) cả hai bộ giải đều dừng ở 16 tàu vì hết ngân sách. Trong lúc dựng đã gặp và sửa các lỗi thật, ví dụ quay lui ở 200 tàu chạy 17 giây nên đổi sang đếm số lần thử, và giao diện lỗi khi xoá tên tàu do dùng kết quả cũ.

Khác biệt giữa hai bản được tóm tắt ở bảng dưới. Các cải tiến là do Claude đề xuất; em chưa tự đánh giá hết giá trị của từng điểm. Chỗ bản đầu tiên làm tốt hơn (cận cắt nhánh mạnh hơn, trang Tổng quan) cũng ghi rõ.

Hạng mục	Bản đầu tiên	Bản nộp
Giới hạn quay lui	400.000 nút hoặc 1,5 giây	Số lần thử (60.000; module 4: 400.000), cùng đầu vào luôn ra cùng kết quả
Cận quay lui module 1	Cộng cả r × thời gian các tàu chưa xếp	Chỉ F × (ω − số hợp đồng đang mở), yếu hơn
Module 2: cặp cấm	Chưa nhập được trên giao diện	Nhập được trên giao diện
Dữ liệu mẫu, nhập/xuất	Mẫu viết tay, chưa nhập/xuất	Mẫu sinh từ bộ testcase; nhập và xuất JSON
Giải thích trên giao diện	Mã giả, bảng ứng viên	Thêm cột độ phức tạp và bảng thuật ngữ
Trang Tổng quan	Có	Không có
Bộ testcase	88 ca	79 ca, sinh lại bằng bộ sinh mới

Các hạn chế của bản nộp được nêu ở mục 8.5.

CHƯƠNG 7. THIẾT KẾ THỰC NGHIỆM

7.1 Mục tiêu

- Kiểm tra **hàm thuật toán** cho kết quả đúng và hợp lệ trên nhiều dạng dữ liệu. Bộ testcase không kiểm lớp API.
- So sánh chất lượng và thời gian giữa các thuật toán của cùng một module.
- Làm lộ các trường hợp mà thuật toán tham lam sai, để demo có chỗ minh hoạ.

7.2 Cách tạo testcase

Testcase được sinh bằng chương trình tools/gen_testcases.py với hạt giống ngẫu nhiên cố định, nên chạy lại cho cùng bộ dữ liệu. Mỗi testcase là một tệp JSON gồm: mã (ví dụ m1-T05), module, nhóm, tiêu đề, mô tả, dữ liệu vào (đúng định dạng nhập vào giao diện) và kỳ vọng (đáp án tối ưu tham chiếu, nguồn của đáp án đó, có hay không có nghiệm). Ba kiểu testcase: (1) mẫu viết tay để minh hoạ một hiện tượng cụ thể (ví dụ bẫy tham lam); (2) ngẫu nhiên theo cỡ và hạt giống; (3) dựng sẵn với đáp án biết trước theo cấu trúc (ví dụ các lô được cắt ra từ các thùng đầy nên tối ưu đã biết).

7.3 Phân nhóm và kích thước

Nhóm	Module 1	Module 2	Module 3	Module 4	Tổng
Thông thường	5	3	4	4	16
Dữ liệu nhỏ	3	3	3	3	12
Biên	8	5	4	3	20
Đặc biệt	6	4	3	3	16
Dữ liệu lớn	5	3	3	4	15
Tổng	**27**	**18**	**17**	**17**	**79**

Bảng 7.1. Số testcase theo nhóm và module

Nhóm	Quy mô	Nội dung được kiểm
Thông thường	3–10 phần tử	Mẫu chính của từng module (ví dụ cái túi kinh điển, First-Fit tệ hơn FFD) và các ca ngẫu nhiên thường gặp
Dữ liệu nhỏ	1–5 phần tử	Các ca nhỏ nhất: một tàu, hai tàu rời nhau, ba tàu chồng nhau, hai cụm tàu cách xa
Biên	1–12 phần tử	F = 0, r = 0, F = r = 0, quỹ 0 giờ; hai tàu chạm đầu mút (rời = đến); tàu qua đêm (giờ > 24); giờ lẻ 0,25; đơn giá bằng 0; thiếu một kỹ năng; yêu cầu nặng đúng bằng quỹ hoặc mọi yêu cầu quá nặng; lô nặng đúng bằng tải trọng; khối lượng lẻ 0,1
Đặc biệt	3–10 phần tử	Bẫy cho tham lam (cái túi, FFD, phân công theo giờ đến); chu trình lẻ C5 và đồ thị đầy đủ K6 do ràng buộc thêm; cặp cấm lặp, đảo chiều, tự thân; vô nghiệm do vi phạm điều kiện Hall; nhiều phần tử giống hệt nhau; hoà về giá trị/giờ
Dữ liệu lớn	12–200 phần tử	Các ca đạt giới hạn tối đa của demo (200 tàu, 60 tàu 12 đội, 60 lô, 25 yêu cầu); ca ngẫu nhiên; ca nối đuôi; ca dựng sẵn với tối ưu biết trước

Bảng 7.2. Mô tả các nhóm testcase

Nhóm "dữ liệu không hợp lệ" mà đề gợi ý ("nếu hệ thống có xử lý") không đưa vào bộ này, vì phạm vi kiểm thử là hàm thuật toán chứ không phải lớp nhận dữ liệu.

7.4 Cách đảm bảo bao phủ các trường hợp

- **Đáp án tham chiếu độc lập với thuật toán:** vét cạn cho ca nhỏ; công thức đóng cho ca đặc biệt (ví dụ F = 0 hay r = 0 làm bài toán suy biến, các tàu nối đuôi); chương trình quy hoạch động riêng cho cái túi cỡ lớn; ca dựng sẵn cho đóng thùng cỡ lớn.
- **Mỗi thuật toán đều được kiểm:** bộ kiểm tra áp dụng cho cả tham lam, DSATUR, quy hoạch động, quay lui. Với kết quả của thuật toán không tối ưu, chỉ yêu cầu hợp lệ và không tốt hơn tối ưu; với thuật toán chính xác thì yêu cầu bằng tối ưu.
- **Bộ kiểm tra tự tính lại:** chi phí và tính hợp lệ (không chồng giờ, đủ kỹ năng, không vượt quỹ, không vượt tải) đều được tính lại từ lời giải, không tin giá trị do thuật toán báo.
- **Ca không có đáp án tham chiếu (6 ca lớn):** chỉ kiểm tính hợp lệ và chi phí tính lại, không so với tối ưu.

7.5 Cách chạy

```
cd replication/app
python3 tools/gen_testcases.py   # sinh lại testcases/m*/
npm run testcases                # chạy, ghi testcases/RESULTS.md và
results.json
npm test                         # pytest, gồm cả 79 testcase
```

Mỗi testcase có thể dán trực tiếp phần dữ liệu vào giao diện tương ứng để xem thuật toán chạy từng bước.

CHƯƠNG 8. ĐÁNH GIÁ

8.1 Tính đúng đắn: kết quả chạy testcase

Tất cả **79/79** testcase đạt (đã so với đáp án tham chiếu và qua bộ kiểm tra độc lập), và 237 bài kiểm thử pytest đều qua. Bảng chi tiết từng ca nằm trong tệp `testcases/RESULTS.md`, sinh tự động bởi `tools/run_testcases.py`.

Module	Thông thường	Nhỏ	Biên	Đặc biệt	Lớn	Đạt / Tổng
1 Thuê bến	5/5	3/3	8/8	6/6	5/5	27/27
2 Phân công đội	3/3	3/3	5/5	4/4	3/3	18/18
3 Quỹ giờ công	4/4	3/3	4/4	3/3	3/3	17/17
4 Xếp container	4/4	3/3	3/3	3/3	4/4	17/17

Bảng 8.1. Kết quả theo nhóm và module

8.2 Chất lượng của các thuật toán

Testcase	Tối ưu tham chiếu	Kết quả các thuật toán	Nhận xét
m1-T05 (8 tàu)	3780	Tham lam 3960; hợp đồng rẻ nhất 3800; DSATUR 3840; quay lui 3780	Bốn kết quả khác nhau
m1-B04 (9 tàu, F = 0)	295	Tham lam 365; hợp đồng rẻ nhất 295; DSATUR 360; quay lui 295	Khi F = 0 chỉ cách xét giờ trống mới tối ưu
m1-L01 (30 tàu)	(không có)	Tham lam 8275; hợp đồng rẻ nhất 7945; DSATUR 8245; quay lui 7945	Quay lui hết ngân sách, bằng hợp đồng rẻ nhất
m2-T03 (6 tàu, 4 đội)	481	Tham lam 485; tàu khó trước 485; quay lui 481	Chỉ quay lui tối ưu
m2-D02 (6 tàu)	198	Tham lam không gán hết; tàu khó trước 243,5; quay lui 198	Tham lam theo giờ đến bị kẹt dù có nghiệm
m3-T04 (cái túi kinh điển)	220	Tham lam 160; QHĐ 220; quay lui 220	Tham lam thiếu 60
m3-L01 (25 yêu cầu)	997	Tham lam 970; QHĐ 997; quay lui 997	Tham lam thiếu 27
m4-T04 (9 lô)	4	FF 5; FFD 4; BFD 4; quay lui 4	FF dùng thừa 1 tàu
m4-D02 (8 lô)	3	FF 4; FFD 4; BFD 4; quay lui 3	Chỉ quay lui tối ưu

Bảng 8.2. Một số ca tiêu biểu so sánh thuật toán

- **Module 3:** tham lam theo giá trị/giờ thấp hơn tối ưu ở 4/17 testcase (thiếu từ 1 đến 60), còn quy hoạch động và quay lui luôn đạt tối ưu.
- **Module 4:** FFD và BFD luôn không kém FF; quay lui đạt tối ưu ở các ca có đáp án tham chiếu. Ở m4-D02 ba thuật toán xếp tuần tự đều dùng 4 tàu trong khi tối ưu là 3, chỉ quay lui tìm ra. Ở m4-L01 (30 lô dựng sẵn) quay lui tìm được 6 tàu đầy tải còn ba thuật toán kia dùng 7.

- **Module 1 và 2:** các thuật toán tham lam thường đạt tối ưu ở ca đơn giản nhưng lệch ở các ca được thiết kế để làm lộ khác biệt; quay lui luôn đạt tối ưu ở các ca có đáp án tham chiếu. Ở các ca lớn không có đáp án (m1-L01, m1-L04, m2-L01, m2-L02), quay lui hết ngân sách và trả về kết quả tốt nhất tìm được.

8.3 Thời gian thực hiện

Số liệu lấy từ cột "ms" trong RESULTS.md (đo trên môi trường chạy kiểm thử, lấy lớn nhất trong các thuật toán của ca đó).

- 74/79 testcase chạy dưới 10 ms; các ca ngoài nhóm dữ liệu lớn đều dưới 4 ms (lớn nhất là 3,7 ms, m3-D03).
- Ca lớn nhất của module 3 (m3-L01, 25 yêu cầu, quỹ 100) chạy trong 3,8 ms cho cả ba thuật toán.
- Các ca lớn có quay lui hoàn tất sớm chạy nhanh: m1-L02 và m1-L03 (100 tàu) khoảng 5 ms, m1-L05 (200 tàu nối đuôi) khoảng 6 ms; m4-L01 (30 lô) mất khoảng 0,47 giây.
- **Bốn ca lớn không kịp chứng minh tối ưu** (m1-L01, m1-L04, m2-L01, m2-L02) hết ngân sách 60.000 lần thử và được đánh dấu "chưa chắc tối ưu"; thời gian chạy từ khoảng 0,36 giây (m1-L01) đến 2,3 giây (m2-L02).

8.4 Giao diện và khả năng minh hoạ từng bước

- **Phát từng bước:** nút Tiếp/Lùi, chạy tự động và thanh trượt; mã giả tô sáng đúng dòng đang chạy; bảng ứng viên kèm lý do bị loại (trùng giờ, thiếu kỹ năng, không vừa tải); thông điệp mô tả từng bước; với quay lui có thêm phương án tốt nhất hiện có và cận.
- Các góc nhìn khác của module 1: biểu đồ Gantt các tàu theo hợp đồng, đồ thị xung đột, bảng so sánh bốn thuật toán (có cột độ phức tạp).
- **Dữ liệu và kết quả:** nhập dữ liệu trực tiếp (module 2 có cả ô nhập cặp tàu cấm chung đội), nạp mẫu có sẵn sinh từ bộ testcase, nhập và xuất JSON; nhãn kết quả "TỐI ƯU", "CHƯA CHẮC TỐI ƯU" hoặc "VÔ NGHIỆM"; bảng thuật ngữ giải thích các từ như "phương án tốt nhất hiện có", "bỏ nhánh".
- **Cách duyệt:** các bước được lưu sẵn nên chuyển bước tức thì, không gọi lại máy chủ.

8.5 Hạn chế

- **Quay lui hết ngân sách** ở bốn ca lớn (m1-L01, m1-L04, m2-L01, m2-L02): kết quả hợp lệ và lặp lại được, nhưng chưa chắc tối ưu. Trên ca m4-L02 của bản đầu tiên (60 lô, tối ưu 15) bộ giải của bản nộp cũng dừng ở 16 tàu, giống bản đầu tiên.
- **Cận quay lui của module 1 yếu hơn bản đầu tiên:** chưa cộng phần r × thời gian của các tàu chưa xếp (mục 4.2.3).
- **Kiểm tra vô nghiệm của module 2 chỉ là điều kiện cần:** chưa xét cặp cấm thêm, nên có ca vô nghiệm mà vẫn phải để quay lui kết luận.
- **Giới hạn kích thước:** 200 tàu (module 1), 60 tàu và 12 đội (module 2), 25 yêu cầu, quỹ 200 giờ và n × (W + 1) ≤ 2500 (module 3), 60 lô (module 4); tối đa 3000 bước mỗi thuật toán.

- **Một số ca lớn** (6 ca) không có đáp án tối ưu tham chiếu nên chỉ kiểm tính hợp lệ và chi phí tính lại.
- **Thời gian chạy** đo trên một môi trường, chưa lặp nhiều lần để lấy trung bình; quay lui của module 2 mất khoảng 1,2 đến 2,3 giây ở các ca 30–60 tàu.
- **Giao diện** không có trang Tổng quan như bản đầu tiên và không còn nút chuyển danh sách tàu từ module 1 sang module 2.

8.6 Hướng phát triển

- Thêm vào cận quay lui của module 1 phần r × thời gian các tàu chưa xếp (như bản đầu tiên) để cắt nhánh mạnh hơn và giảm số ca hết ngân sách.
- Mở rộng kiểm tra vô nghiệm của module 2 để xét cả cặp tàu cấm chung đội.
- Thêm trang Tổng quan và nút chuyển danh sách tàu từ module 1 sang module 2.
- Bổ sung kiểm tra tự động làm hỏng lời giải một cách có chủ đích để chắc rằng bộ kiểm tra độc lập từ chối lời giải sai.
- Bổ sung thêm thuật toán so sánh (ví dụ nhánh cận có sắp xếp tốt hơn) và đo thời gian trên nhiều lần chạy.

8.7 Tổng hợp mối liên hệ và kết luận

Bài toán	Mô hình	Cấu trúc dữ liệu	Thuật toán	Demo	Testcase / đánh giá
Thuê bến	Tô màu đồ thị khoảng, chi phí F + r·khoảng phủ	Danh sách kề, mảng màu, lo/hi/cnt của hợp đồng	Tham lam, hợp đồng rẻ nhất, DSATUR, quay lui	Phát từng bước, biểu đồ Gantt, đồ thị xung đột	27 ca, đạt 27/27
Phân công đội	Gán việc có danh sách cho phép	Danh sách đội đủ kỹ năng, tập xung đột, cận suffix	Tham lam (2 kiểu), quay lui, kiểm tra Hall	Bảng ứng viên có lý do loại	18 ca, đạt 18/18
Quỹ giờ công	Cái túi 0/1	Bảng dp, mảng take	Tham lam, QHĐ, quay lui + cận phân số	Bảng dp từng ô, truy vết	17 ca, đạt 17/17
Xếp container	Đóng thùng	Mảng gán bin, mảng tải loads	FF, FFD, BFD, quay lui	Thùng xếp theo từng lô	17 ca, đạt 17/17

Bảng 8.3. Mối liên hệ bài toán – mô hình – cấu trúc dữ liệu – thuật toán – demo – testcase

Đồ án đã xây dựng được một demo cho bốn bài toán điều phối cảng, mỗi bài toán có nhiều thuật toán để so sánh và quan sát từng bước. Bộ 79 testcase với kiểm tra độc lập cho thấy các thuật toán chạy đúng, và cho thấy rõ chỗ thuật toán tham lam sai, qua đó minh hoạ được vì sao cần quy hoạch động và quay lui. Hạn chế lớn nhất còn lại là quay lui hết ngân sách ở vài ca lớn nên kết quả ở đó chưa chắc tối ưu.

TÀI LIỆU THAM KHẢO

[1] Huỳnh Thị Thanh Thương, bài giảng CS112 (Phân tích và thiết kế thuật toán), Trường Đại học Công nghệ Thông tin: quy trình giải quyết vấn đề và các chiến lược thiết kế, khung tham lam, khung quay lui try(i), quy hoạch động, ví dụ tô màu đồ thị.

[2] D. Brélaz, "New methods to color the vertices of a graph", Communications of the ACM 22(4), 1979, tr. 251–256.

[3] J. Kleinberg, É. Tardos, Algorithm Design, Addison-Wesley, 2006: `4.1 (Interval Scheduling, Interval Partitioning),` 6.4 (Subset Sums and Knapsacks).

[4] T. Cormen, C. Leiserson, R. Rivest, C. Stein, Introduction to Algorithms, 3rd ed., MIT Press, 2009: §16.2.

[5] G. B. Dantzig, "Discrete-variable extremum problems", Operations Research 5(2), 1957, tr. 266–288.

[6] S. Martello, P. Toth, "Lower bounds and reduction procedures for the bin packing problem", Discrete Applied Mathematics 28, 1990, tr. 59–70.

[7] D. S. Johnson, A. Demers, J. D. Ullman, M. R. Garey, R. L. Graham, "Worst-case performance bounds for simple one-dimensional packing algorithms", SIAM Journal on Computing 3(4), 1974, tr. 299–325.

[8] M. R. Garey, D. S. Johnson, Computers and Intractability, W. H. Freeman, 1979.

[9] P. Hall, "On representatives of subsets", Journal of the London Mathematical Society 10(1), 1935, tr. 26–30.

[10] H. W. Kuhn, "The Hungarian method for the assignment problem", Naval Research Logistics Quarterly 2, 1955, tr. 83–97.

[11] M. C. Golumbic, Algorithmic Graph Theory and Perfect Graphs, Academic Press, 1980.

[12] GitHub Spec Kit, kho mã và tài liệu: https://github.com/github/spec-kit (truy cập 13/10/2026).

[13] Superpowers (Jesse Vincent), kho mã: https://github.com/obra/superpowers (truy cập 13/10/2026).

Công nghệ sử dụng: Python (FastAPI, uvicorn, pydantic, pytest, Playwright), React 18.3 và Vite 5.4. Tên và phiên bản cụ thể nằm trong `replication/app/backend/requirements.txt` và `replication/app/frontend/package.json`.

PHỤ LỤC A. HƯỚNG DẪN CHẠY VÀ CẤU TRÚC MÃ NGUỒN

A.1 Chạy trên Codespaces hoặc trên máy

Mã nguồn: github.com/SerenaVo-code/berth-contract-coloring (bản nộp nằm trong thư mục replication/app). Trên GitHub: Code → Codespaces → Create codespace on main. Môi trường chỉ có sẵn Python và Node, không tự cài hay tự chạy gì. Trong Terminal:

```
cd replication/app
npm run setup    # cài thư viện (một lần)
npm run dev      # chạy backend :8000 + giao diện :5173 (Ctrl+C để dừng)
```

Mở tab Ports, bấm biểu tượng địa cầu ở cổng 5173. Trên máy cá nhân cũng dùng đúng các lệnh này.

A.2 Cấu trúc thư mục

```
(đường dẫn tính từ replication/app)
backend/core.py       ghi bước, ngân sách số lần thử
backend/berth.py      module 1: thuê bến
backend/assign.py     module 2: phân công đội
backend/knapsack.py   module 3: quỹ giờ công (cái túi)
backend/binpack.py    module 4: xếp container (đóng thùng)
backend/server.py     FastAPI (một endpoint cho mỗi module)
frontend/src/         giao diện React
tests/                pytest + kịch bản Playwright
testcases/m*/         79 testcase (4 module × 5 nhóm)
testcases/RESULTS.md  bảng kết quả chạy testcase
tools/                gen_testcases.py, run_testcases.py, checker.py,
cross_check.py
```

PHỤ LỤC B. NHẬT KÝ PROMPT (PHIÊN 12–13/10/2026)

Cột 2 là tin nhắn gốc đúng như đã gửi (viết nhanh, bằng tiếng Việt). Cột 3 là bản tiếng Anh đã chỉnh cho rõ ràng; bản này chỉ diễn đạt lại, không thêm yêu cầu nào em chưa nói. Phiên ngày 10/10 không còn lưu tin nhắn gốc; bằng chứng là bốn commit nêu ở Bảng 6.2.

#	Tin nhắn gốc	Bản tiếng Anh đã chỉnh	AI đã làm	Em quyết định / kiểm tra
1	Văn bản đã dán.txt ấy, đề đó	Here is the instructor's assignment brief (the pasted text file). Please use it as the project requirements.	Đọc đề, liệt kê các sản phẩm phải nộp	Cung cấp đề, xác định là yêu cầu chuẩn
2	viết test case giúp mình	Please write a test case suite for the project.	Sinh bộ testcase 4 module và bộ chạy độc lập (commit 149fc94)	—
3	không test case ở đâu là test cái hàm thể hiện thuật toán thôi	The test cases should only test the functions that implement the algorithms, not the API layer. Please change the suite accordingly.	Bỏ nhóm kiểm API; bộ chạy gọi thẳng hàm giải (88 ca, commit 3a5da29)	Chỉnh phạm vi của AI: chỉ kiểm hàm thuật toán
4	còn gì liên quan tới code cần cải thiện không nhỉ	Is there anything else about the code that should be improved?	Rà soát và đề xuất danh sách cải thiện	Chọn việc nào làm, việc nào để sau
5	`e2e.png` (ảnh chụp thử) đang nằm trong repo, nên xoá. oke	The test screenshot e2e.png is in the repository and should be removed. Please remove it.	Xoá ảnh, thêm .gitignore, sửa script e2e ghi vào thư mục tạm (commit 9c39abd, e82fa6f)	Đồng ý xoá
6	ủa tại sao / vì sao testcase lại sinh ra ảnh để phải xóa	Why was it there? Why does running the tests produce an image that has to be deleted?	Giải thích ảnh do script e2e sinh ra và AI lỡ dùng git add -A	Hỏi nguyên nhân thay vì chấp nhận ngay
7	thôi, xem kĩ thử còn gì cải thiện không, tại thấy cô yêu cầu về code cũng nhieuf, prompt rồi framework đồ	Please review the project carefully for anything still missing. The brief asks for a lot about the code, the prompts and the framework.	Đối chiếu dự án với đề, liệt kê chỗ thiếu	Nhận ra đề yêu cầu ghi lại prompt và framework
8	làm phần 4 giúp mình	Please do item 4 of your proposed list.	Thực hiện mục 4 trong danh sách đề xuất	—
9	ok thôi nguồn từ từ, còn giù nũa	OK, let's leave the sources for later. What else is left to do?	Tạm dừng phần nguồn, đề xuất việc còn lại	Quyết định tự kiểm chứng nguồn sau
10	(tin nhắn dài bằng tiếng Anh, xem bên dưới)	Kept as sent (see below).	Soạn tài liệu Word 15 trang về prompt dựng lại	Đặt ràng buộc: ghi rõ là dựng lại, không nhận đã dùng framework, không sửa code, không cài công cụ

#	Tin nhắn gốc	Bản tiếng Anh đã chỉnh	AI đã làm	Em quyết định / kiểm tra
11	umm, cơ mà đảm bảo rằng để cô thấy được phần mình làm nó sẽ nhiều hơn, đừng để cô nghĩ mình giao toàn bộ việc cho AI làm hen	Please make sure the document shows clearly what I did myself, so the instructor does not think I handed all the work to the AI.	Thêm mục "Vai trò của tác giả" vào tài liệu	Đối chiếu mục này với những gì mình thực sự đã làm

Bảng B.1. Các prompt của phiên 12–13/10/2026

Tin nhắn số 10, nguyên văn

I now want to prepare a retrospective document showing how prompts could be structured using an AI development framework such as Superpowers or Spec Kit. Please help me create a Word document containing a realistic set of reconstructed prompts based on the completed project's functionality, architecture, and implementation. Requirements: Briefly review Superpowers and Spec Kit, then select the framework whose workflow best fits the project. Use the existing project and its completed implementation as context to reconstruct plausible prompts for key development stages, such as requirement analysis, code exploration, solution design, implementation, and testing. Make the prompts technically realistic and consistent with the project's actual architecture and business logic. For each prompt, explain its intended purpose, the expected AI output, and how its structure follows the chosen framework's principles. Include an assessment of the prompt's potential effectiveness, with specific strengths and possible weaknesses. Organise the material into a professional Word document suitable for review by my instructor. Keep the document concise and focused on prompt quality rather than explaining the frameworks in excessive detail. Important: These are retrospective reconstructions, not actual historical prompts. Clearly label them as reconstructed examples and do not claim that the framework was used during the original implementation. Do not modify the codebase, implement features, or install any tools. This task is documentation only.

PHỤ LỤC C. TÀI LIỆU PROMPT DỰNG LẠI THEO FRAMEWORK

Tài liệu đính kèm "Reconstructed Prompts for the Port Coordination Demo" (15 trang) gồm 10 prompt được dựng lại sau khi dự án hoàn thành, theo quy trình của Spec Kit. Mỗi prompt có mục đích, kết quả AI dự kiến, cách cấu trúc bám theo framework, điểm mạnh, điểm yếu, điểm em phải tự quyết định hoặc kiểm tra, và xếp hạng. **Đây là ví dụ dựng lại, không phải prompt lịch sử, và dự án này không dùng Spec Kit hay Superpowers.**

Mã	Giai đoạn	Nội dung prompt
P1	Phân tích yêu cầu	Nguyên tắc dự án (constitution)
P2	Phân tích yêu cầu	Đặc tả tính năng, chưa nói công nghệ
P3	Phân tích yêu cầu	Làm rõ các điểm mơ hồ
P4	Khảo sát mã	Khảo sát chỉ đọc module 1
P5	Thiết kế giải pháp	Kế hoạch kỹ thuật cho module 2–4
P6	Thiết kế giải pháp	Danh sách việc có thứ tự, kiểm chứng được
P7	Thiết kế giải pháp	Kiểm tra nhất quán giữa các tài liệu
P8	Cài đặt	Cài đặt giai đoạn module 2
P9	Kiểm thử	Bộ testcase với đáp án tham chiếu độc lập
P10	Kiểm thử	Kiểm tra hội tụ dựa trên bằng chứng

Bảng C.1. Danh sách prompt trong tài liệu bổ trợ